

Full SmolVLA: Autoregressive Foundation VLM Policy

SmolVLM-256M-Instruct Backbone for Robotic Manipulation

Autonomous Robotics & VLA Research Lab

https://github.com/krazykarthik2/manual_vla

September 2026

1. Abstract

Full SmolVLA integrates the complete HuggingFace SmolVLM-256M-Instruct foundation model as a native perception backbone. Unlike SmolVLA's dual-stream frozen CLIP approach, Full SmolVLA feeds images and text into a unified autoregressive vision-language model and taps multi-layer intermediate representations for rich multimodal conditioning of the flow matching action decoder.

2. Architecture Overview

Full SmoIVLA: SmoIVLM-256M Foundation Architecture

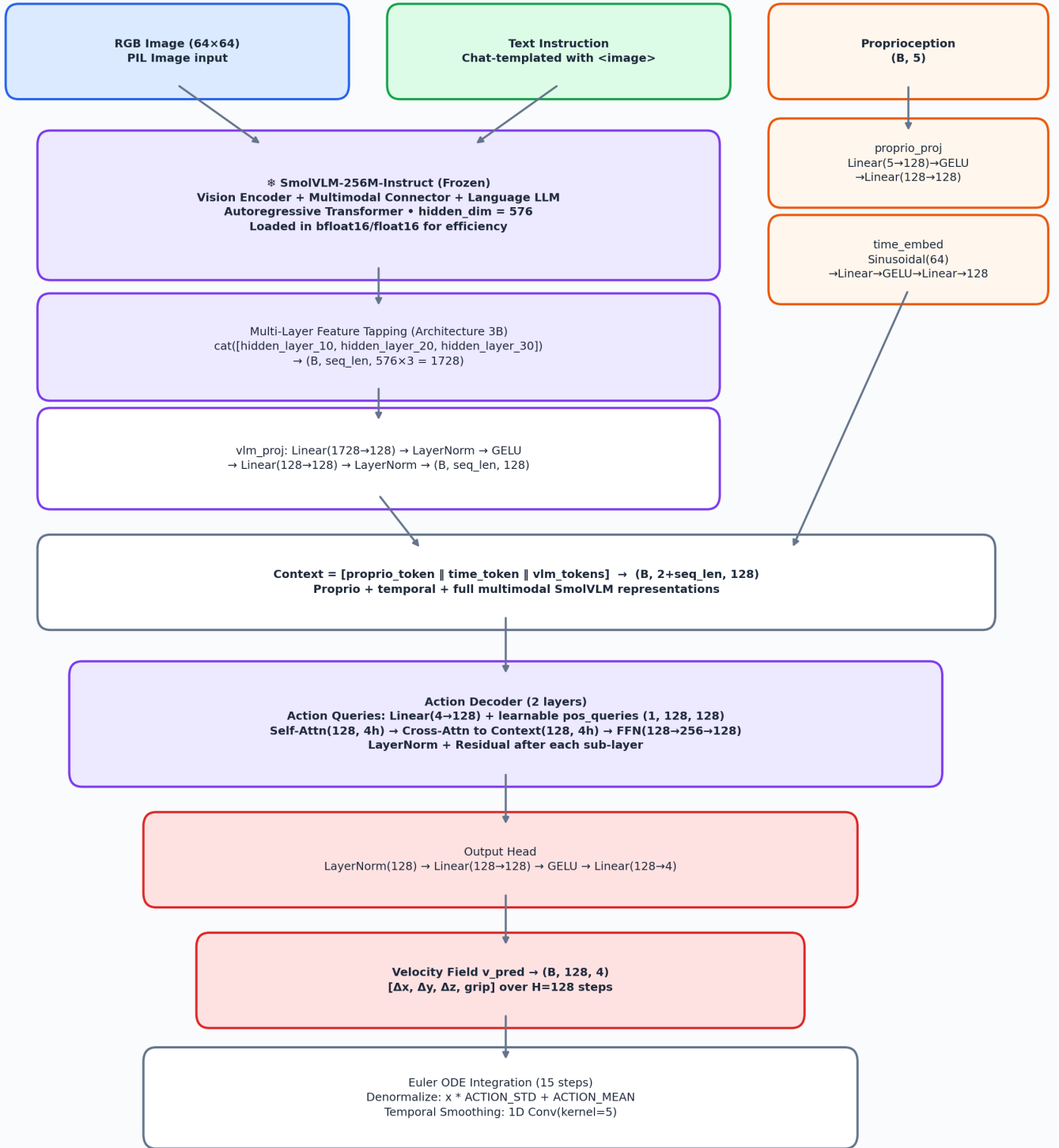


Figure 1: Full SmoIVLA architecture using SmoIVLM-256M-Instruct backbone.

Full SmoIVLA consists of three major components:

1. SmoIVLM-256M-Instruct: frozen autoregressive vision-language backbone
2. Multi-layer intermediate feature projection
3. Cross-attention action decoder with flow matching

3. SmoIVLM Foundation Backbone

3.1 Model Details

Model: HuggingFaceTB/SmolVLM-256M-Instruct

Type: SmolVLMForConditionalGeneration (autoregressive VLM)

Hidden dimension: 576

The backbone is loaded in reduced precision:

- bfloat16 on CUDA if BF16 is supported
- float16 on CUDA otherwise
- float32 on CPU

All backbone weights are frozen (requires_grad=False).

3.2 Input Formatting

Images and text are formatted using HuggingFace's chat template:

```
messages = [{"role": "user", "content": [  
    {"type": "image"},  
    {"type": "text", "text": prompt}  
]}]
```

The processor tokenizes text and encodes images into the model's expected format with special <image> tokens embedded in the token sequence.

3.3 Multi-Layer Intermediate Feature Tapping (Architecture 3B)

Instead of using only the final hidden state, Full SmolVLA taps three intermediate layers of the SmolVLM transformer:

- Layer 10: Early spatial and edge features
- Layer 20: Mid-level relational features
- Layer 30: High-level goal and semantic features

The hidden states from these three layers are concatenated along the feature dimension:

```
multi_layer_hidden = cat([h_10, h_20, h_30], dim=-1)  
Shape: [B, seq_len, 576 * 3] = [B, seq_len, 1728]
```

This multi-scale representation captures both fine spatial detail and abstract semantic understanding.

4. Multimodal Context Projection

The concatenated multi-layer features are projected to the action model dimension:

```
vlm_proj = Sequential(  
    Linear(1728 -> 128),  
    LayerNorm(128),  
    GELU(),
```

```
Linear(128 -> 128),  
LayerNorm(128)  
)
```

Output: [B, seq_len, 128] multimodal context tokens.

5. Conditioning Signals

5.1 Proprioception Encoding

The 5D proprioceptive state (end-effector position + gripper) is projected:

```
Linear(5 -> 128) -> GELU -> Linear(128 -> 128)
```

Produces a single proprioceptive token [B, 1, 128].

5.2 Continuous Sinusoidal Time Embedding

The diffusion timestep t in $[0, 1]$ is encoded via sinusoidal frequencies:

```
freqs = exp(-log(10000) * arange(32) / 31)  
time_emb = cat([sin(t * freqs), cos(t * freqs)], dim=-1) ->  $\mathbb{R}^{64}$ 
```

Then projected:

```
Linear(64 -> 128) -> GELU -> Linear(128 -> 128)
```

Produces a single time token [B, 1, 128].

5.3 Full Context Assembly

The conditioning context is assembled by concatenation:

```
context = [proprio_token, time_token, vlm_tokens]  
Shape: [B, 2 + seq_len, 128]
```

This provides the action decoder with proprioceptive grounding, temporal awareness, and rich multimodal scene understanding.

6. Cross-Attention Action Decoder

The action decoder generates 128-step trajectories for 4 action dimensions (dx, dy, dz, grip). It consists of 2 decoder layers, each with:

6.1 Action Query Initialization

Noisy action inputs x_t are projected to $d_{\text{model}}=128$ and combined with learnable positional queries:

```
act_queries = Linear(4 -> 128)(x_t) + pos_queries  
pos_queries: Parameter(1, 128, 128) initialized with  $N(0, 0.02)$ 
```

6.2 Decoder Layer Structure (x2)

Each layer performs three operations:

1. Self-Attention: act_queries attend to themselves
MultiheadAttention(128, 4 heads) + LayerNorm + residual
2. Cross-Attention: act_queries attend to the full context
MultiheadAttention(128, 4 heads) + LayerNorm + residual
This is where the action trajectory is grounded in visual/language understanding.
3. Feed-Forward: 2-layer MLP
Linear(128 -> 256) -> GELU -> Linear(256 -> 128) + LayerNorm + residual

6.3 Output Head

The final action predictions are produced by:

LayerNorm(128) -> Linear(128 -> 128) -> GELU -> Linear(128 -> 4)

Output: predicted velocity field v_pred of shape [B, 128, 4].

7. Flow Matching Trajectory Generation

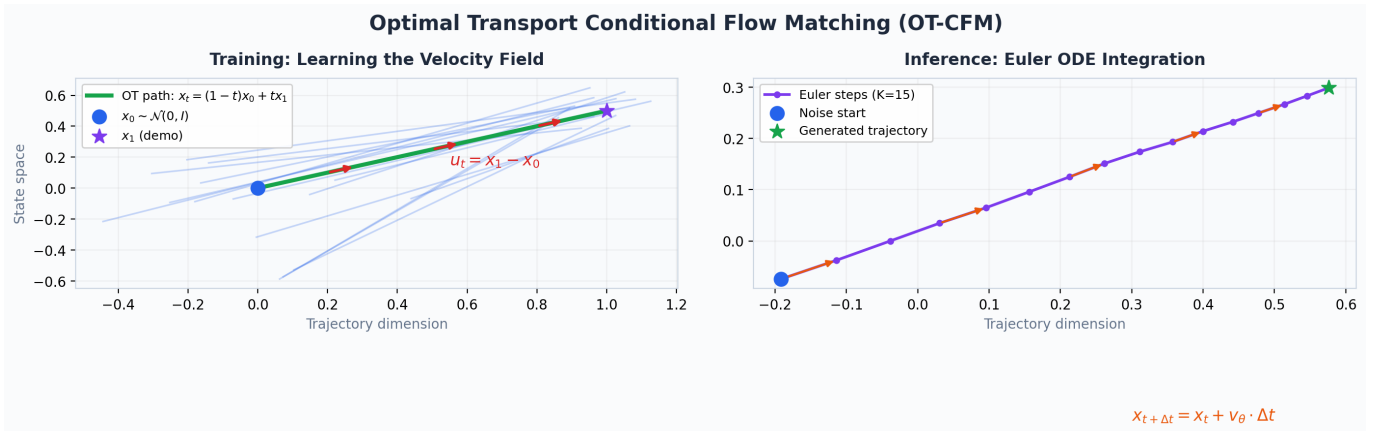


Figure 2: Optimal Transport Conditional Flow Matching dynamics.

Training:

$x_t = (1-t)x_0 + tx_1$, $x_0 \sim N(0, I)$, x_1 = demo trajectory

Target velocity: $u_t = x_1 - x_0$

Loss weights: $w = [1.2, 1.2, 1.5, 2.5]$ for [x, y, z, grip]

Inference (sampling):

1. Start from noise: $x_0 \sim N(0, I)$, shape [B, 128, 4]
2. Euler integration over $K=15$ steps: $x_{t+dt} = x_t + v(x_t, t, context) \cdot dt$
3. Denormalize: $raw = x \cdot ACTION_STD + ACTION_MEAN$
4. Temporal smoothing: 1D convolution with kernel size 5

8. Activation Returns for Visualization

When `return_activations=True`, the decoder returns a dictionary of intermediate layer outputs for visualization and debugging:

- `context`: full conditioning sequence [B, 2+seq_len, 128]
- `l1_sa`, `l1_ca`, `l1_out`: Layer 1 self-attn, cross-attn, FFN outputs
- `l2_sa`, `l2_ca`, `l2_out`: Layer 2 outputs
- `ca_w1`, `ca_w2`: Cross-attention weight matrices [B, 128, context_len]
- `v_pred`: Final velocity prediction [B, 128, 4]

These activations enable spectrograms of layer-wise processing and cross-attention grounding maps.